CS1101S / PA2 QUICK REFERENCE

Source Academy Python §3 | Original practice patterns | Check the actual assessment rules

CONTRACT FIRST

Representation? Output? May mutate? Must restore? No new pairs? Initial-call or total cost? Empty/singleton? Sorted/disjoint? State a meaning for every helper argument.

Shapes: llist(a,b) = pair(a,pair(b,None)). pair(a,b) is a record. Linked tail is data; stream tail is a nullary function. Never head/tail(None).

SOURCE ACADEMY PYTHON §3

Use x=x+1, separate assignments, for i in range(n), explicit arguments. No .append/.pop, slices, tuples/unpacking, comprehensions, default parameters or chained comparisons. Conditions must be Boolean: is_none(xs), i<n. Use print(...) to inspect results.

b=a aliases; b=tail(b) rebinds; set_tail(b,x) mutates. is tests identity; == compares values/structure. nonlocal rebinds an enclosing local. / divides; // floors; ** is power. Check bounds before indexing.

```
mat = [None] * R
for r in range(R):
    mat[r] = [0] * C  # fresh each time
# NOT [[0] * C] * R: shared rows
```

Indexed lists have allocated slots: track a used length. append(xs,ys) is LINKED concatenation. len(a) is for indexed lists/strings; length(xs) traverses a linked list.

LISTS: BASE / STEP / COMBINE

```
def map_ll(f, xs):
    if is_none(xs): return None
    return pair(f(head(xs)),
                map_ll(f, tail(xs)))
```

Filter: build rest once; prepend head if kept, otherwise return rest. Prepending to an accumulator reverses order. Repeated linked append/length may cost O(n^2).

ADJACENT PAIRS / DIRECTED ROUTE

```
if is_none(xs) or is_none(tail(xs)):
    return 0
u = head(xs)
v = head(tail(xs))
return mat[u][v] + walk(tail(xs))
```

Fragment: walk closes over mat. Advance ONE node. n values give n-1 legs. mat[u][v] need not equal mat[v][u].

ENCODE / DECODE STATE

Encoder: emit first unchanged; thereafter emit current-previous, then previous=current ORIGINAL input. Decoder: updated=total+head(ds); emit updated and pass updated onward. Empty gives None.

```
def scan(xs, total):
    if is_none(xs): return None
    new = total + head(xs)
    return pair(new, scan(tail(xs), new))
# scan(xs, 0): no extra initial output
```

MATRICES / INDEX FORMULAS

mat[r][c]; rows=len(mat), cols=len(mat[0]). Make independent rows. Separate block offset from direction within the block.

```
# Alternating columns, starting at 1:
offset = r if c % 2 == 0 else R-1-r
value = c*R + 1 + offset
# Alternating rows:
offset = c if r % 2 == 0 else C-1-c
value = r*C + 1 + offset
# Transpose: out[c][r] = mat[r][c]
```

Test 2x4, 1xC and Rx1. O(RC) construction.

POSTFIX: FIXED SLOTS + LIVE COUNT

```
stack = [None] * len(tokens)
top = 0
for i in range(len(tokens)):
    token = tokens[i]
    if is_string(token):
        top = top - 1
        right = stack[top]
        top = top - 1
        left = stack[top]
        # Compute value = left OP right.
        stack[top] = value
    else:
        stack[top] = token
    top = top + 1
return stack[0]
```

Template: add operator dispatch as in G10. Assumes valid binary tokens. top counts live values; stale slots do not matter. First removed = RIGHT. Unary op consumes one. O(tokens) time/space. Test - and /.

DESTRUCTIVE REVERSE

```
prev = None
while not is_none(xs):
    nxt = tail(xs)  # save before mutation
    set_tail(xs, prev)
    prev = xs
    xs = nxt
return prev
```

No new pairs or head changes. Filter/partition: save next, detach node, link using first/last references. Terminate final tails at None.

DESTRUCTIVE ASCENDING MERGE

```
def merge(xs, ys):
    if is_none(xs): return ys
    if is_none(ys): return xs
    if head(xs) <= head(ys):
        set_tail(xs, merge(tail(xs), ys))
        return xs
    set_tail(ys, merge(xs, tail(ys)))
    return ys
```

Sorted, acyclic, disjoint inputs. <= takes left first on ties. O(n+m) time AND stack: recursion is not tail-position. Merge sort: find midpoint, SAVE second=tail(slow), CUT set_tail(slow,None), sort halves, merge. O(n log n); O(n) stack with this merge.

CS1101S / PA2 QUICK REFERENCE

Source Academy Python §3 | Original practice patterns | Check the actual assessment rules

BACKTRACK: GUARD / MARK / EXPLORE / UNDO

```
def shortest(maze, r, c):
    R = len(maze)
    C = len(maze[0])
    if r<0 or r>=R or c<0 or c>=C:
        return math_inf
    if maze[r][c] == '#': return math_inf
    if maze[r][c] == 'G': return 0
    old = maze[r][c]
    maze[r][c] = '#'
    best = min(shortest(maze,r-1,c),
               shortest(maze,r+1,c),
               shortest(maze,r,c-1),
               shortest(maze,r,c+1))
    maze[r][c] = old
    return 1 + best
```

Mark only the current path. Restore BEFORE return, including success. Goal=0 counts moves, not cells. 1+infinity=infinity. Worst-case exponential time; O(RC) stack.

Objective	Fail	Goal	Combine
Reachable	False	True	OR
Simple path count	0	1	Sum
Minimum moves	infinity	0	1+min

STATE MUST EXPLAIN FUTURE CHOICES

Key: [r,c,has_key]. Wall break: [r,c,used]. Exact k moves with revisits: [r,c,k]. Same coordinate with different resources can be a different state.

Do not cache path-marked DFS only by coordinate: blocked cells depend on history. Exact-k walks with revisits can cache by [r,c,k]; test success at k==0 if earlier goal visits may continue.

BFS: process distance layers; mark on enqueue. A preallocated queue of RC slots uses front/back indices. Each cell enqueues once. Expanded states need expanded capacity. O(RC) for unit-cost grid shortest paths. Weighted moves need different reasoning.

COST / BINARY-SEARCH BOUNDARY

Pair operations, indexed access and len: O(1). Linked length/index: O(n). Preallocate k slots: O(k). Count repeated work. Output, control stack and closures are separate space costs.

```
# First index with a[i] >= x; else len(a)
lo = 0
hi = len(a)
while lo < hi:
    mid = (lo + hi) // 2
    if a[mid] < x: lo = mid + 1
    else: hi = mid
return lo
```

Before lo is too small; at/after hi meets threshold. Empty/duplicates work. O(log(n+1)) time, O(1) space. §3 proper tail calls use O(1) control space; pending addition/pair construction does not.

STREAM = pair(value, THUNK)

```
def chars(text):
    def go(i):
        if i == len(text): return None
        return pair(text[i], lambda: go(i+1))
    return go(0)
```

head(s)=value; tail(s)=function; stream_tail(s)=next stream. Recursive call INSIDE lambda. Carry an index. O(1) construction; O(n) for n outputs. Filter may scan far before its first match.

MEMOIZED TAILS: DEFINE IF NOT SUPPLIED

```
def memo_fun(f):
    done = False
    value = None
    def force():
        nonlocal done, value
        if not done:
            value = f()
            done = True
        return value
    return force
```

pair(x,memo_fun(lambda: next_step(...))). done distinguishes cached None/0. Repeated forcing shares the suffix. Retaining n generated nodes costs O(n) memory.

LAZY SCAN / CURSORS

```
def scan_stream(s, total):
    if is_none(s): return None
    new = total + head(s)
    return pair(new, memo_fun(
        lambda: scan_stream(stream_tail(s),new)))
```

stream(a,b,c) is a finite constructor. eval_stream(s,n) requires n available elements. Walk one cursor: repeated stream_ref(s,i) from the head costs O(n^2) traversal even with cached tails. Avoid an unnecessary final force.

Kolakoski: seed 1,2,2; next length cursor at element 2, next symbol 1. Emit length copies; advance length cursor ONCE per run; toggle 3-symbol. Memoize seed and generated tails. Reader must only demand already available lengths. G18 gives the complete stream construction.

SCOPE / TREES / CPS / EVALUATORS

Closure = function + enclosing bindings. Rebinding an outer local needs nonlocal; mutating its list does not. A new factory call gives fresh state. Return helper results from outer functions.

Tree cases: None / leaf / pair. Process head AND tail. Right fold: op(a,op(b,z)); left fold: op(op(z,a),b). Order matters.

CPS: k is the rest of computation. Base -> k(0); next continuation is lambda rest: k(x*x+rest). §3 tail calls: O(1) control space, but O(n) pending closures here.

Evaluator: inspect tag, recurse, lookup nearest binding. let RHS uses OLD environment; body uses extended one. Closure captures its creation environment. Evaluate only the chosen conditional branch.

FINAL CHECK

Empty/singleton; negatives/repeats; asymmetric/nonsquare; - and /; start=goal/unreachable; restoration; same nodes/no cycles; laziness; repeated forcing; missing return; actual time and space.